



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	ARIANNE BEVERLY MARQUEZ
Section:	AN41
Professor:	MS. CRISOLA TAN
Date:	9/25/26

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.

```

Welcome × MARQUEZ_Technical1.py
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Technical1.py > ...
1 import agentpy as ap
2 import random
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5
6 # ----- Get user input from console -----
7 num_agents = int(input("Enter number of agents: "))
8 grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
9 num_steps = int(input("Enter number of steps: "))
10
11 # ----- Agent Definition -----
12 class RandomWalker(ap.Agent):
13     def step(self):
14         direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
15         x, y = self.position
16         # Keep agents inside grid
17         x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
18         y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
19         self.position = (x, y)
20
21 # ----- Model Definition -----
22 class RandomWalkModel(ap.Model):
23     def setup(self):
24         # Create agents
25         self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
26
0
```

```
Welcome X MARQUEZ_Technical1.py
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Technical1.py > ...
22 class RandomWalkModel(ap.Model):
23     def setup(self):
24
25         # Initialize random positions
26         for agent in self.agents:
27             agent.position = (random.randint(0, self.p.grid_size[0]-1),
28                               random.randint(0, self.p.grid_size[1]-1))
29
30         # Create grid (optional)
31         self.grid = ap.Grid(self, self.p.grid_size, torus=False)
32         self.grid.add_agents(self.agents)
33
34     def step(self):
35         for agent in self.agents:
36             agent.step()
37
38 # ----- Parameters from user input -----
39 parameters = {
40     'agents': num_agents,
41     'grid_size': (grid_size, grid_size),
42     'steps': num_steps
43 }
44
45 # ----- Run Model -----
46 model = RandomWalkModel(parameters)
47 model.setup()
48
49
50
```

0

```

Welcome X MARQUEZ_Technical1.py
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Technical1.py > ...

51 # ----- Interactive Animation -----
52 fig, ax = plt.subplots()
53 ax.set_xlim(0, model.p.grid_size[0])
54 ax.set_ylim(0, model.p.grid_size[1])
55 ax.set_xticks(range(model.p.grid_size[0]+1))
56 ax.set_yticks(range(model.p.grid_size[1]+1))
57 ax.grid(True)
58
59 scat = ax.scatter([], [], s=200) # s = size of agents
60
61 def update(frame):
62     model.step()
63     x = [agent.position[0] for agent in model.agents]
64     y = [agent.position[1] for agent in model.agents]
65     scat.set_offsets(list(zip(x, y)))
66     return scat,
67
68 ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
69 plt.show()

```

- 0
3. Enter the following values when prompted:
- **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

```

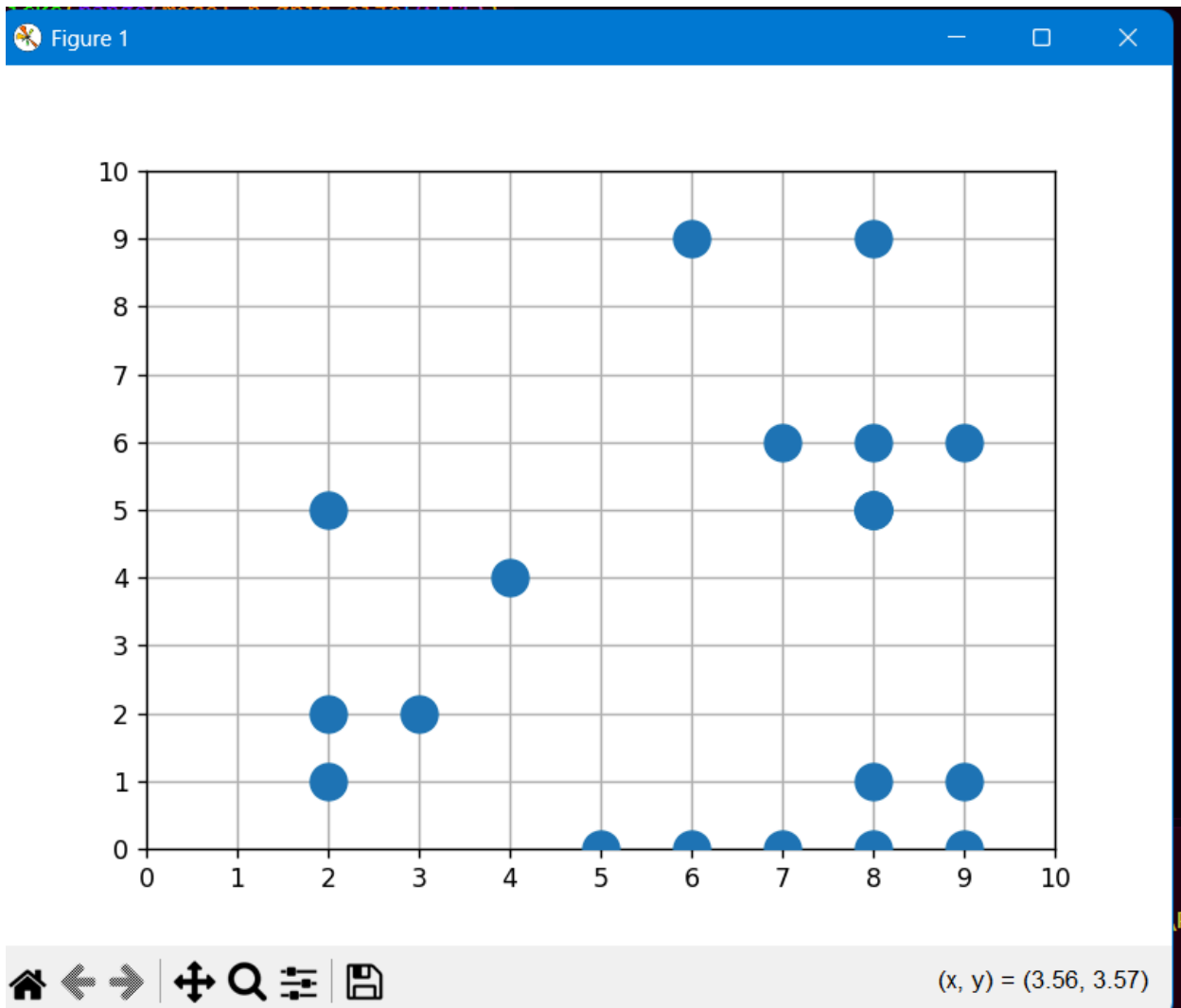
Problems Output Debug Console Terminal Ports
PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Yani\AppData\Local\Programs\Python\Python313\python.exe c:/Python/CS0065/TECHNICALS/MARQUEZ_Technical1.py
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 30

```

```

Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 30

```



These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

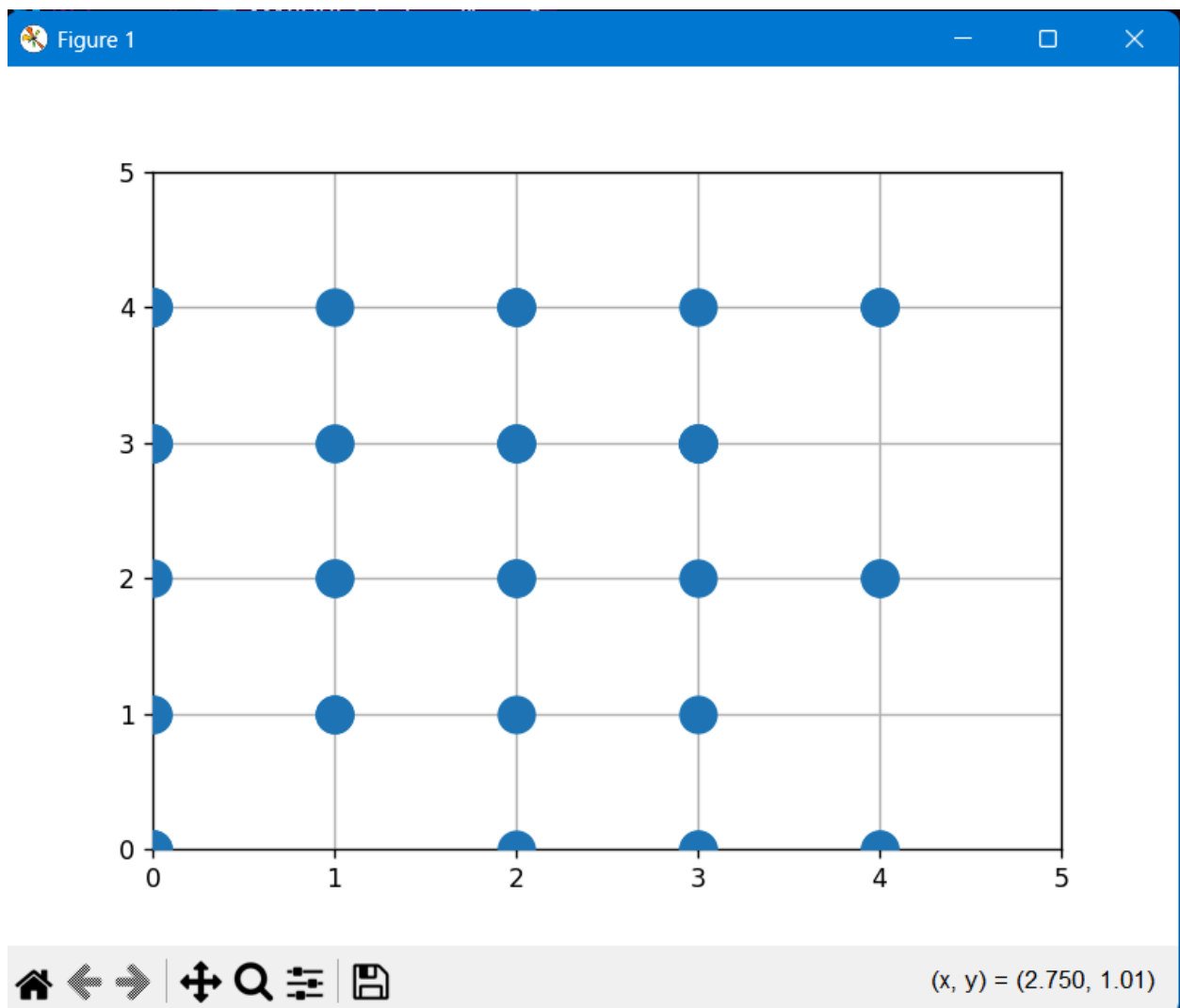
Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)

```
Enter number of agents: 50
Enter grid size (e.g., 10 for 10x10): 5
Enter number of steps: 25
```



- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↓

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

print('\nProgrammed by: Arianne Beverly V. Marquez\n')

class CustomWalker(ap.Agent):
    def setup(self):
        # 1. Track agent paths: Initialize a list to store
        # coordinates
        self.path = []

    def step(self):
        # 2. Change movement rules: Introduce a biased
        # behavior preference
        if self.model.p.behavior == 'biased':
            choices = [(1,0), (1,0), (0,1), (0,1), (-1,0),
            (0,-1)]
        else:
            choices = [(1,0), (-1,0), (0,1), (0,-1)]

        direction = random.choice(choices)
        x, y = self.position

        x = max(0, min(self.model.p.grid_size[0]-1, x +
        direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y +
        direction[1]))
        self.position = (x, y)

        # Append current position after moving
        self.path.append(self.position)
```

```

class WalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents,
CustomWalker)
        for agent in self.agents:
            agent.position = (random.randint(0,
self.p.grid_size[0]-1),
                                random.randint(0,
self.p.grid_size[1]-1))
            agent.path.append(agent.position) # Store
starting point

    def step(self):
        self.agents.step()

num_agents = int(input('Enter number of agents: '))
grid_size = int(input('Enter grid size (e.g., 10 for
10x10): '))
num_steps = int(input('Enter number of steps: '))

# 4. Compare scenarios: Define parameters
modell1 = WalkModel({'agents': num_agents, 'grid_size':
(grid_size, grid_size), 'behavior': 'random'})
modell2 = WalkModel({'agents': num_agents, 'grid_size':
(grid_size, grid_size), 'behavior': 'biased'})
modell1.setup()
modell2.setup()

# --- INTERACTIVE ANIMATION ---
fig_anim, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))
fig_anim.suptitle('Agent Movement Comparison (Close window
when done to see analysis)')

ax1.set_title('Scenario 1: Random Walk')
ax2.set_title('Scenario 2: Biased Walk')

for ax in [ax1, ax2]:
    ax.set_xlim(0, grid_size)
    ax.set_ylim(0, grid_size)
    ax.grid(True)

scat1 = ax1.scatter([], [], s=100, c='blue', zorder=2)
scat2 = ax2.scatter([], [], s=100, c='red', zorder=2)

```

```

# Create line objects for the path trails
lines1 = [ax1.plot([], [], alpha=0.3, c='blue',
zorder=1)[0] for _ in range(num_agents)]
lines2 = [ax2.plot([], [], alpha=0.3, c='red', zorder=1)[0]
for _ in range(num_agents)]

def update(frame):
    model1.step()
    model2.step()

    x1 = [a.position[0] for a in model1.agents]
    y1 = [a.position[1] for a in model1.agents]
    scat1.set_offsets(list(zip(x1, y1)))

    x2 = [a.position[0] for a in model2.agents]
    y2 = [a.position[1] for a in model2.agents]
    scat2.set_offsets(list(zip(x2, y2)))

    # Update trail lines
    for i, agent in enumerate(model1.agents):
        lines1[i].set_data([p[0] for p in agent.path],
[p[1] for p in agent.path])

    for i, agent in enumerate(model2.agents):
        lines2[i].set_data([p[0] for p in agent.path],
[p[1] for p in agent.path])

    return [scat1, scat2] + lines1 + lines2

ani = animation.FuncAnimation(fig_anim, update,
frames=num_steps, blit=True, repeat=False)
plt.show()

# --- 3. FINAL POSITION ANALYSIS ---
# This window will pop up automatically after you close the
animation window
fig_hist, (ax3, ax4) = plt.subplots(1, 2, figsize=(12, 5))
fig_hist.suptitle('Final Position Analysis')

final_x1 = [a.position[0] for a in model1.agents]
final_y1 = [a.position[1] for a in model1.agents]
ax3.hist2d(final_x1, final_y1, bins=[range(grid_size + 1),
range(grid_size + 1)], cmap='Blues')

```

```
ax3.set_title('Scenario 1 Final Distribution')

final_x2 = [a.position[0] for a in model2.agents]
final_y2 = [a.position[1] for a in model2.agents]
ax4.hist2d(final_x2, final_y2, bins=[range(grid_size + 1),
range(grid_size + 1)], cmap='Reds')
ax4.set_title('Scenario 2 Final Distribution')

plt.show()
```

Problems Output Debug Console **Terminal** Ports

PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> & C

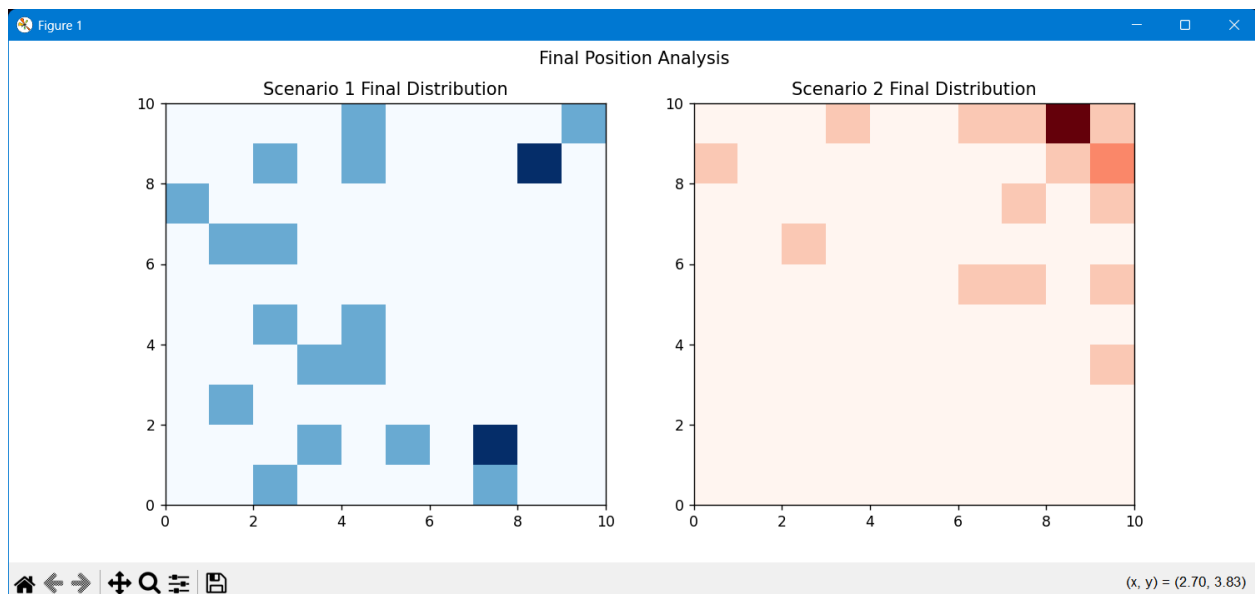
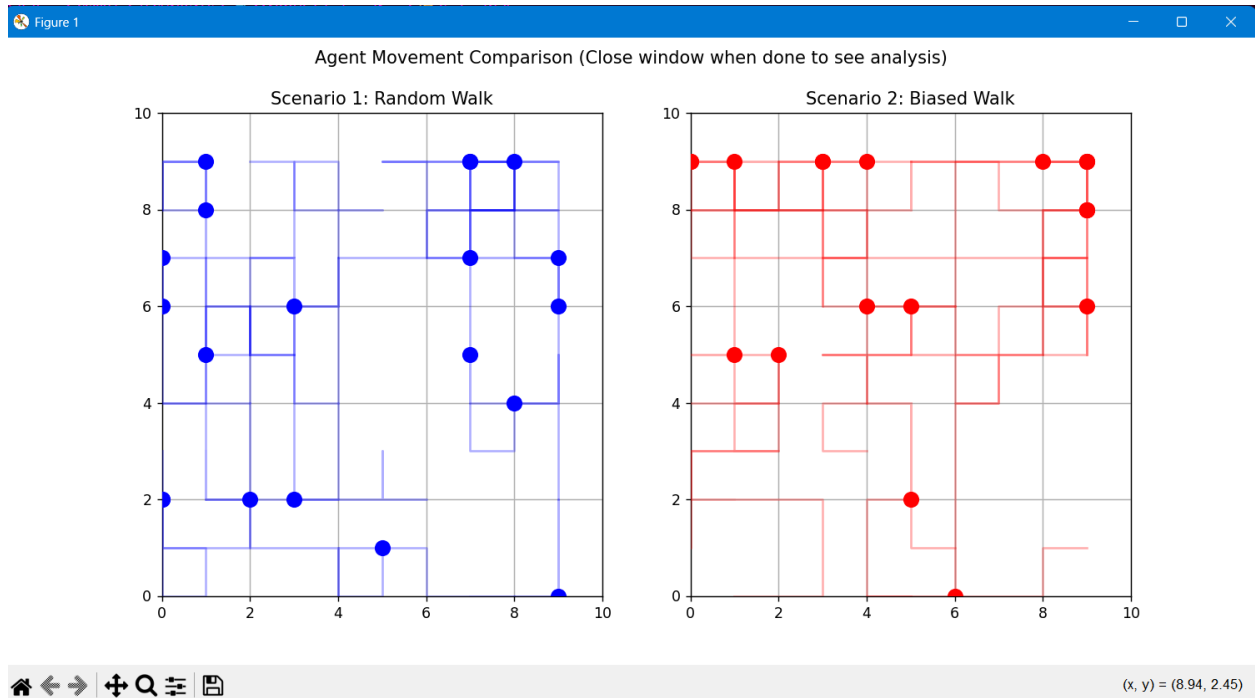
Programmed by: Arianne Beverly V. Marquez

Enter number of agents: 20

Enter grid size (e.g., 10 for 10x10): 10

Enter number of steps: 30

PS C:\Users\Yani\AppData\Local\Programs\Microsoft VS Code> █

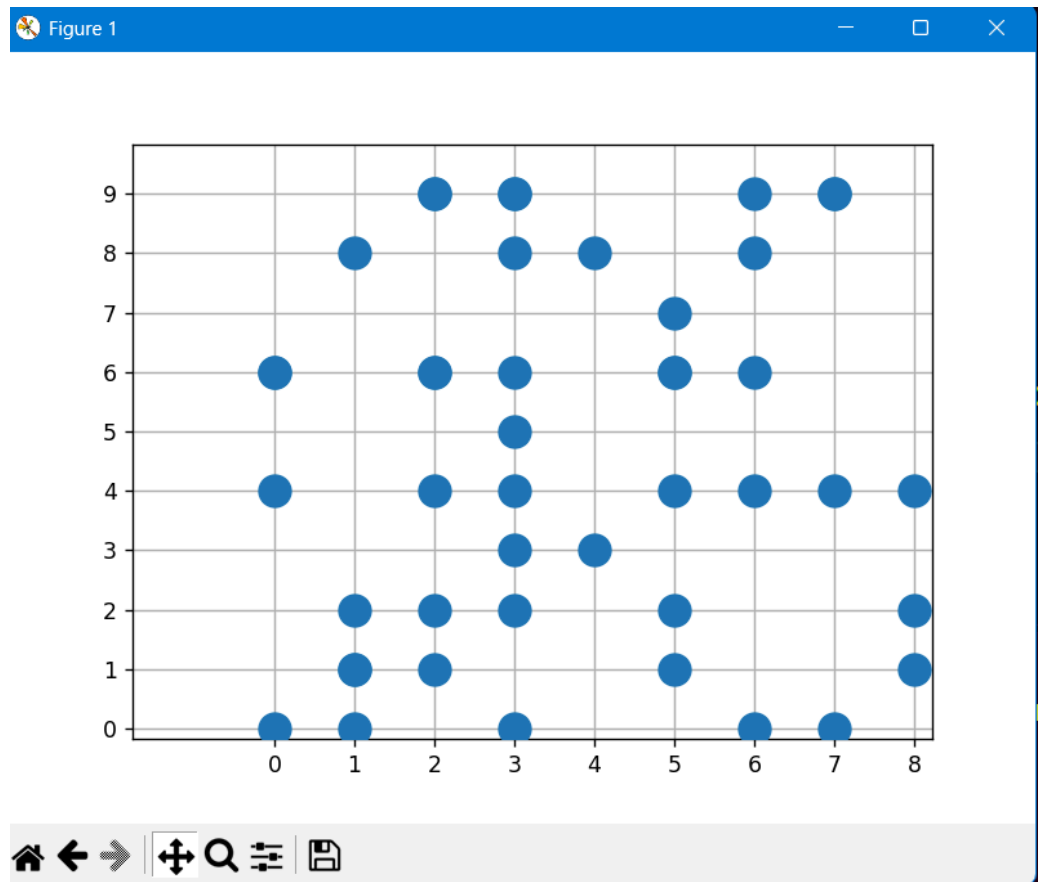


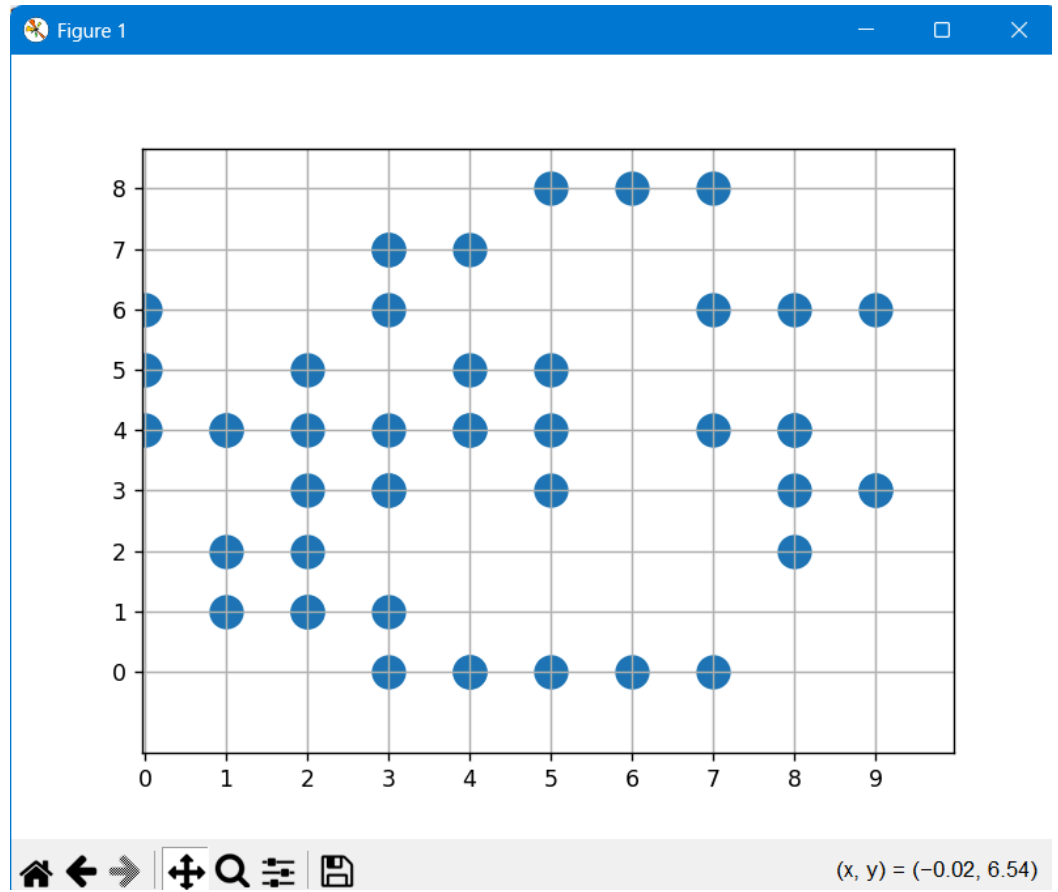
Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

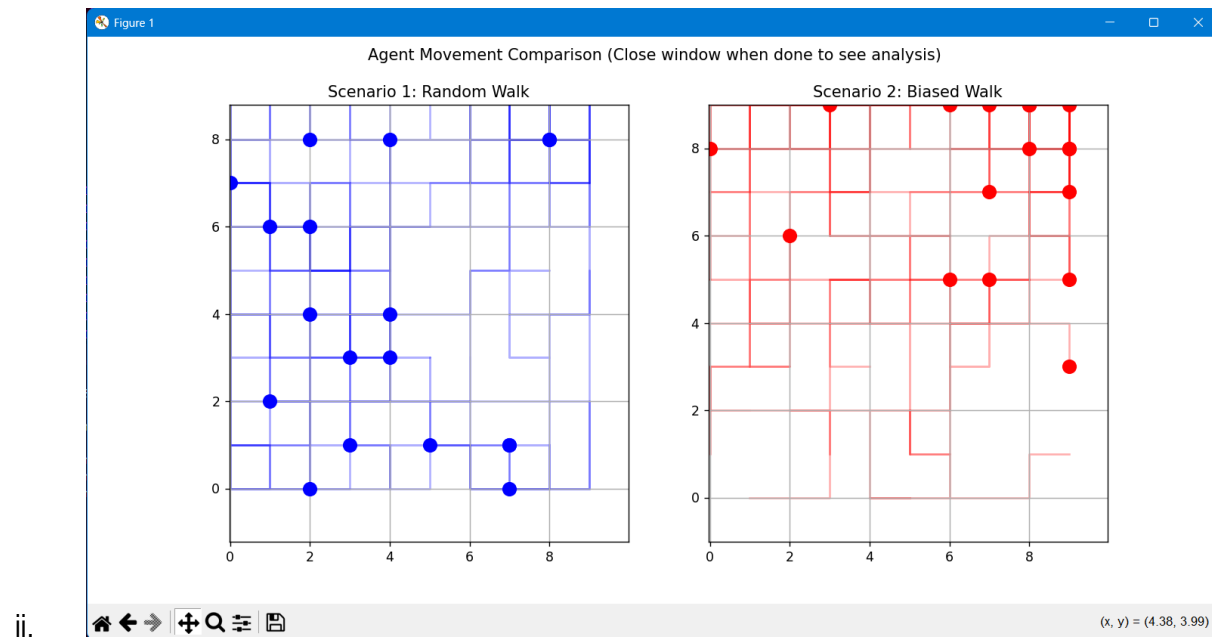
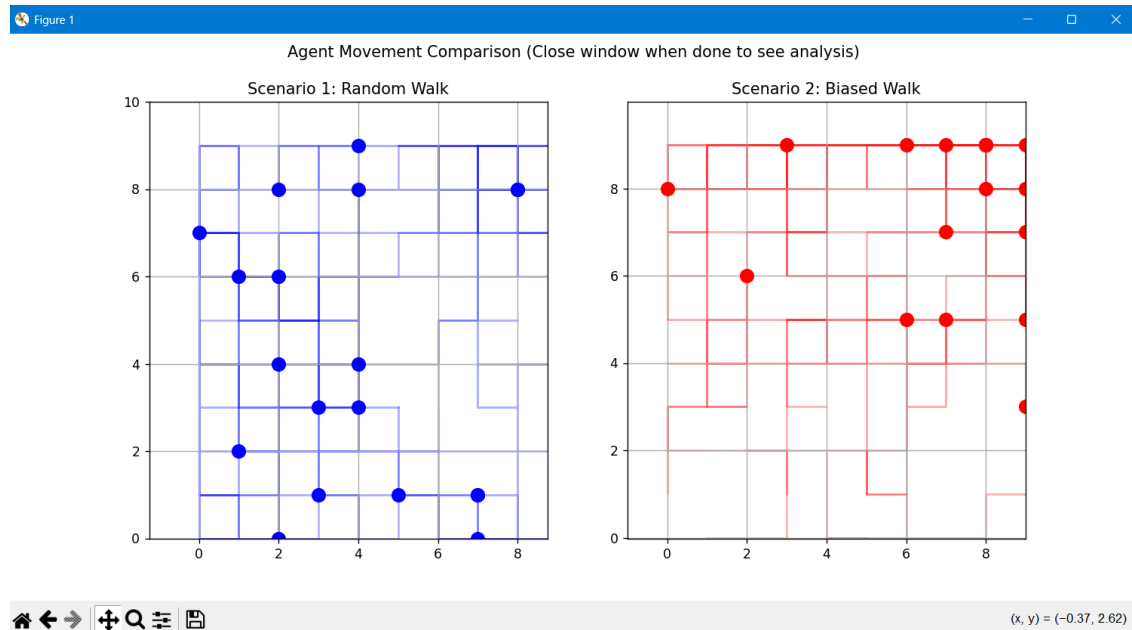
Answer the Questions ↓

1. When the number of agents are very high compared to the grid size, some of the circles (the agents) are only displayed in half size in the program.
2. The increasing number of steps affect the spread of agents in terms of how quick they move. As per what I have observed, the higher the number of steps are, the quicker they move from one place to another. The lower the steps are, the slower they move.
3. My answer is divided into two regarding the agents moving outside of the grid:
 - a. In the Initial Code, they do not move outside the grid, but they do overlap from the line.





- b. In the Modified code, the agents remained on the line despite the high number of agents and number of steps.



4. I think this simulation can be compared in crowded events like **in transportation terminals during rush hours, in malls, in concerts and tourist spots.** I think the circles/agents represent how people move back and forth in certain locations.